

SCA TME 6 AES masqué selon le schéma de Herbst

Delmotte Aurélien

Zhang Xiru

September 25, 2025

Contents

1	Introduction	2
2	Implémentation	2
2.1	Masquage du State lors du Chiffrement	2
2.2	Masquage de la Génération des Clés de Ronde	3
3	Attaque CPA sur l'Implémentation Masquée	5
4	Attaque Template sur l'Implémentation Masquée	5
4.1	Construction des Templates	6
4.2	Exécution de l'Attaque	6
5	Conclusion	6

1 Introduction

L'objectif de ce travail est d'implémenter le schéma de masquage de Herbst pour l'AES. Ce schéma de masquage comprend deux composantes principales: le masquage de l'état lors du chiffrement et le masquage de la génération des clés de ronde. Suite à l'implémentation de ces techniques de masquage, l'efficacité de la protection offerte par ce schéma a été testée à travers deux approches d'attaque: une attaque par analyse de puissance corrélative (CPA) et une attaque par template. Ces tests permettent d'évaluer la résistance de l'implémentation masquée d'AES face à des techniques avancées d'extraction de clés, fournissant ainsi une mesure de l'efficacité des masques appliqués.

2 Implémentation

2.1 Masquage du State lors du Chiffrement

Cette partie implique la modification de la fonction `aes()` ainsi que des fonctions qu'elle appelle. L'objectif est de masquer les données traitées durant les différentes étapes du chiffrement AES, en particulier lors des opérations de `SubBytes`, `ShiftRows` et `MixColumns`, pour brouiller les liens entre les entrées et les sorties et ainsi compliquer les analyses de canaux latéraux.

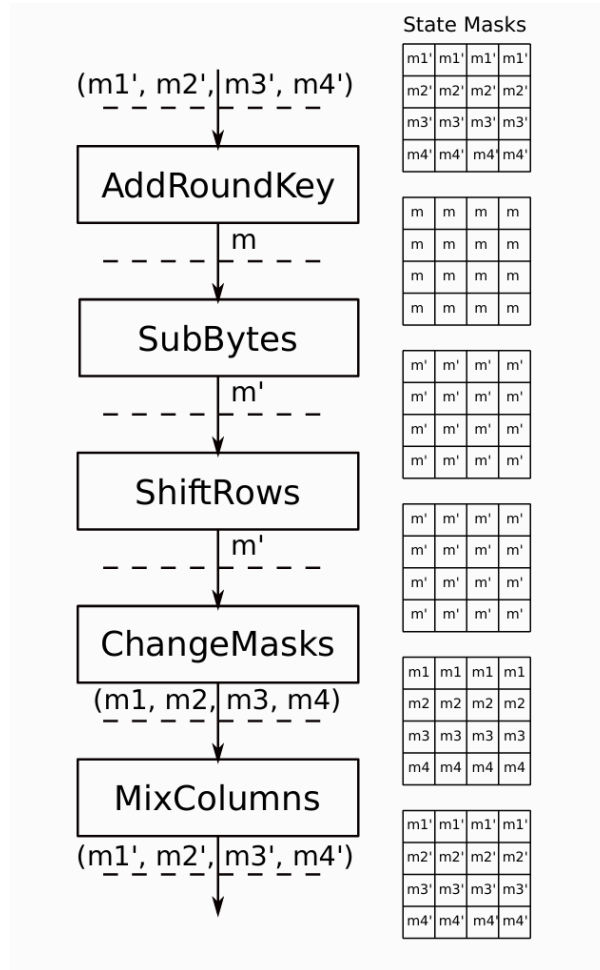


Figure 1: Caption

Pour implémenter le masquage de l'état dans le processus de chiffrement AES, nous devons ajouter des masques à chaque étape selon 1.

Initialisation et application des masques de ligne Avant le début des rondes de chiffrement, chaque élément de la matrice d'état est d'abord masqué par une opération XOR avec le masque de ligne

correspondant $(m1', m2', m3', m4')$.

```

1 for(int i = 0; i < 4; i++) {
2     for(int j = 0; j < 4; j++) {
3         x[i][j] ^= masp[i];
4     }
5 }

```

SBox masquée La SBox masquée est calculée selon la formule:

$$SBox'[x \oplus m] = SBox[x] \oplus m'$$

où x est le byte original avant la substitution, m est le masque appliqué à l'index de la SBox, et m' est le masque appliqué à la valeur sortie de la SBox. Cette approche permet de déguiser la relation directe entre les entrées et les sorties de la SBox originale lors de l'exécution de l'algorithme.

```

1 uint8_t sboxp[256];
2 for(int i=0;i<256;i++) sboxp[i^m] = sbox[i]^mp;

```

ChangeMasks L'objectif principal de la fonction **ChangeMasks** est de mettre à jour les masques de l'état après l'opération ShiftRows et avant MixColumns. Ce changement est crucial pour maintenir l'intégrité des masques à travers les différentes étapes de transformation linéaire du chiffrement AES.

Juste après les étapes de SubBytes et ShiftRows, où les masques sont initialement définis à m' pour chaque byte, la fonction **ChangeMasks** ajuste chaque masque à de nouvelles valeurs $m1, m2, m3, m4$ qui sont spécifiquement conçues pour les opérations de MixColumns.

```

1 for(int i=0;i<4;i++){
2     for(int j=0;j<4;j++){
3         x[i][j] ^= mpmask[i];
4     }
5 }

```

L'absence de MixColumns dans le dernier tour signifie également qu'il n'est pas nécessaire de mettre à jour ou de changer les masques utilisés dans le processus de chiffrement.

2.2 Masquage de la Génération des Clés de Ronde

Pour cette section, la fonction **key_schedule** doit être modifiée afin de masquer les clés utilisées durant les différentes rondes du chiffrement.

Initialisation et application du masque initial La fonction commence par appliquer un masque initial aux premières clés directement après leur chargement en mémoire. Ceci est réalisé en exécutant une opération XOR entre les clés initiales et un masque pré-déterminé, noté m .

Traitement des masques pendant l'expansion de clés Durant l'expansion de clés, chaque nouvelle clé dérivée est temporairement stockée dans une variable volatile pour prévenir toute optimisation de compilation qui pourrait omettre des étapes cruciales. Pour les clés qui dépendent de transformations non linéaires comme l'utilisation de la SBox, des masques sont réintroduits après chaque application de la SBox masquée pour garantir que les valeurs résultantes restent protégées.

```

1 void key_schedule_masked(uint8_t key[16], uint8_t round_key[176], uint8_t masp[4], uint8_t
2     m, uint8_t mp, uint8_t sboxp[256]) {
3     for (int32_t i = 0; i < 4; i += 1) {
4         round_key[i * 4] = key[i * 4] ^ (m^masp[0]);
5         round_key[i * 4 + 1] = key[i * 4 + 1] ^ (m^masp[1]);
6         round_key[i * 4 + 2] = key[i * 4 + 2] ^ (m^masp[2]);
7         round_key[i * 4 + 3] = key[i * 4 + 3] ^ (m^masp[3]);
8     }
9     for (int32_t i = 4; i < 40; i += 1) {
10        volatile uint8_t temp[4];
11        for (int32_t j = 0; j < 4; j += 1) {
12            temp[j] = round_key[(i - 1) * 4 + j];
13        }
14        if (i % 4 == 0) {

```

```

14     uint8_t k = temp[0];
15     temp[0] = temp[1];
16     temp[1] = temp[2];
17     temp[2] = temp[3];
18     temp[3] = k;
19
20     temp[0] = sboxp[temp[0]^masp[1]];
21     temp[1] = sboxp[temp[1]^masp[2]];
22     temp[2] = sboxp[temp[2]^masp[3]];
23     temp[3] = sboxp[temp[3]^masp[0]];
24
25     temp[0] = temp[0] ^ rcon[(i / 4) - 1];
26
27     temp[0] = round_key[(i - 4) * 4 + 0] ^ temp[0];
28     temp[1] = round_key[(i - 4) * 4 + 1] ^ temp[1];
29     temp[2] = round_key[(i - 4) * 4 + 2] ^ temp[2];
30     temp[3] = round_key[(i - 4) * 4 + 3] ^ temp[3];
31     round_key[i * 4 + 0] ^=temp[0];
32     round_key[i * 4 + 1] ^=temp[1];
33     round_key[i * 4 + 2] ^=temp[2];
34     round_key[i * 4 + 3] ^=temp[3];
35 } else {
36     volatile uint8_t temp2[4];
37     temp2[0]=round_key[(i - 4) * 4 + 0]^m;
38     temp2[1]=round_key[(i - 4) * 4 + 1]^m;
39     temp2[2]=round_key[(i - 4) * 4 + 2]^m;
40     temp2[3]=round_key[(i - 4) * 4 + 3]^m;
41     temp[0]^=masp[0];
42     temp[1]^=masp[1];
43     temp[2]^=masp[2];
44     temp[3]^=masp[3];
45     round_key[i * 4 + 0] = temp2[0] ^ temp[0];
46     round_key[i * 4 + 1] = temp2[1] ^ temp[1];
47     round_key[i * 4 + 2] = temp2[2] ^ temp[2];
48     round_key[i * 4 + 3] = temp2[3] ^ temp[3];
49 }
50 }
51 for(int i=40;i<44;i++){
52     uint8_t temp[4];
53     for (int32_t j = 0; j < 4; j += 1) {
54         temp[j] = round_key[(i - 1) * 4 + j];
55     }
56     if (i % 4 == 0) {
57         uint8_t k = temp[0];
58         temp[0] = temp[1];
59         temp[1] = temp[2];
60         temp[2] = temp[3];
61         temp[3] = k;
62
63         temp[0] = sboxp[temp[0]^masp[1]];
64         temp[1] = sboxp[temp[1]^masp[2]];
65         temp[2] = sboxp[temp[2]^masp[3]];
66         temp[3] = sboxp[temp[3]^masp[0]];
67
68         temp[0] = temp[0] ^ rcon[(i / 4) - 1];
69     }
70     round_key[i * 4 + 0] = round_key[(i - 4) * 4 + 0] ^ temp[0];
71     round_key[i * 4 + 1] = round_key[(i - 4) * 4 + 1] ^ temp[1];
72     round_key[i * 4 + 2] = round_key[(i - 4) * 4 + 2] ^ temp[2];
73     round_key[i * 4 + 3] = round_key[(i - 4) * 4 + 3] ^ temp[3];
74 }
75 round_key[40 * 4 + 0] ^=(m^masp[0]);
76 round_key[40 * 4 + 1] ^=(m^masp[1]);
77 round_key[40 * 4 + 2] ^=(m^masp[2]);
78 round_key[40 * 4 + 3] ^=(m^masp[3]);
79 round_key[42 * 4 + 0] ^=(m^masp[0]);
80 round_key[42 * 4 + 1] ^=(m^masp[1]);
81 round_key[42 * 4 + 2] ^=(m^masp[2]);
82 round_key[42 * 4 + 3] ^=(m^masp[3]);
83 }

```

La répétition de certaines opérations et l'utilisation de la variable **volatile** sont des techniques employées pour prévenir les optimisations du compilateur qui pourraient autrement éliminer ou modifier l'ordre

des opérations critiques. Ces techniques garantissent que chaque étape de la manipulation des clés est effectivement exécutée tel que spécifié, préservant ainsi les propriétés nécessaires pour le masquage contre les analyses de canaux latéraux.

3 Attaque CPA sur l'Implémentation Masquée

Nous avons identifié la nouvelle localisation des opérations de SubBytes de la première itération dans les traces masquées, ce qui est crucial pour l'analyse de la résistance aux CPA. La figure 2 montre la section des traces où la phase de SubBytes est censée se produire après l'application du masquage, nous avons défini un ensemble de 1000 échantillons pour chaque trace, avec un point de départ (offset) fixé à 2458. Dans le contexte de l'AES masqué, identifier les Points d'Intérêt (POI) pour réaliser une attaque

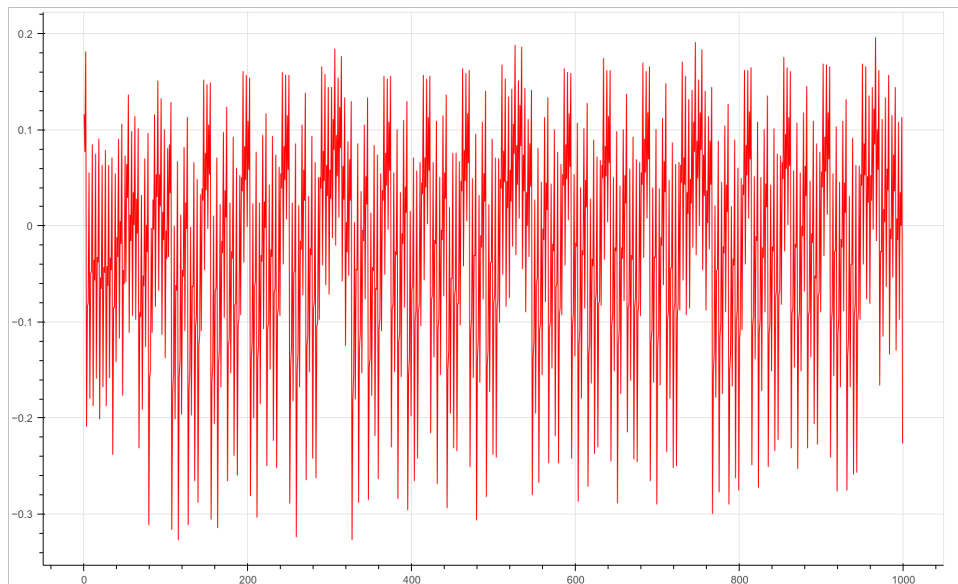


Figure 2: Traces de la phase de SubBytes après application du masquage.

CPA efficace devient plus complexe en raison de la dispersion des traces due au masquage. Toutefois, nous avons adapté notre approche en utilisant une version améliorée de l'algorithme CPA, qui permet un calcul en ligne et optimise le processus en réduisant le besoin de multiples passages sur les données. Malgré nos efforts pour implémenter une attaque par analyse de puissance corrélative (CPA) contre une implémentation AES masquée, nous n'avons pas réussi à retrouver la clé correcte.

```
correct key: 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 9 cf 4f 3c
CPA online: c2 51 36 bd 37 14 23 eb e2 bc c3 b8 27 4f ba c4
Error in byte 0: 0xc2 != 0x2b
```

Figure 3: Attaque CPA.

4 Attaque Template sur l'Implémentation Masquée

Les attaques par template sont une méthode sophistiquée d'analyse de canaux cachés, exploitant la consommation énergétique des dispositifs cryptographiques pour en déduire les clés secrètes. Cette technique repose sur deux phases principales : la construction de templates et leur utilisation pour attaquer un dispositif cryptographique.

La construction des templates implique la collecte de traces de consommation énergétique, dénommées traces T0, lors de l'exécution du dispositif avec des clés et des plaintextes aléatoires. Chaque trace est analysée pour identifier des points d'intérêt (POIs) qui révèlent des informations potentielles sur les opérations cryptographiques internes, principalement autour de la sortie des SBox. Les données collectées sont ensuite utilisées pour établir des modèles statistiques, en supposant que la consommation pour

chaque poids de Hamming de la valeur intermédiaire ciblée suit une distribution normale multivariée. L'attaque proprement dite utilise ces templates pour évaluer et scorer des traces collectées sous une clé constante inconnue, appelées traces T1, en calculant la probabilité que les consommations mesurées correspondent aux différents modèles établis pendant la phase de construction. La clé avec le score cumulatif le plus élevé est identifiée comme la clé secrète du dispositif.

- T0 contient des clés et des plaintextes aléatoires.
- T1 contient une clé constante inconnue et des plaintextes associés.

L'expérience a impliqué l'utilisation de 10000 traces pour construire les templates et 100 traces pour réaliser l'attaque, avec une sélection de 5 points d'intérêt pour une analyse détaillée. En tout, 8200 échantillons ont été nécessaires pour capturer la première itération de l'AES.

4.1 Construction des Templates

La construction des templates est une étape cruciale dans l'analyse de canaux cachés. Pour ce faire, nous avons collecté des traces énergétiques lors de l'exécution de l'algorithme cryptographique avec des clés connues. Chaque trace est analysée pour identifier des points d'intérêt où les variations de consommation énergétique sont susceptibles de révéler des informations sur les valeurs intermédiaires du calcul cryptographique. Les caractéristiques statistiques de ces points, notamment les moyennes et les covariances, sont calculées et utilisées pour construire un modèle de comportement du dispositif cryptographique. Ce modèle est ensuite affiné pour servir de base à l'attaque par template.

4.2 Exécution de l'Attaque

L'attaque par template utilise les templates statistiques pour analyser de nouvelles traces obtenues avec une clé secrète inconnue. En appliquant les modèles de consommation énergétique préétablis à ces traces, nous évaluons la probabilité de chaque valeur possible de l'octet de clé. Pour chaque trace, un score est calculé pour toutes les valeurs possibles de l'octet, basé sur la correspondance entre la consommation mesurée et les prédictions du template. L'octet de clé générant le score le plus élevé à travers toutes les traces est identifié comme étant la partie de la clé la plus probable. Ce processus révèle non seulement la vulnérabilité de l'algorithme cryptographique aux attaques par canaux cachés mais offre également un moyen précis de compromettre la sécurité de l'information cryptée.

```
Correct Key:
# Key : [ 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c ]
Key Guess:
# Key : [ 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c ]
# Nb. bytes found: 16
```

Figure 4: Résultat de l'attaque template

5 Conclusion

L'implémentation des masques dans l'AES augmente considérablement la résilience contre les attaques par analyse de puissance corrélative (CPA) traditionnelles. Par contre, les attaques par template ont prouvé leur efficacité, soulignant la nécessité de prendre en compte des méthodes d'attaque plus avancées lors de la conception de systèmes cryptographiques sécurisés.

En conclusion, cette étude confirme que bien que le masquage puisse empêcher les attaques simples, des approches sophistiquées comme les attaques par template restent une menace viable.